

Saga Pattern - Cheat Sheet

The Saga pattern solves a hard problem in microservices: how do you complete a multi-step business operation that touches several services, when any step can fail and you can't use a distributed transaction?

The answer is to split the work into a sequence of local transactions. Each step has a compensating action that undoes it. If a later step fails, you run compensations in reverse to roll back what already happened.

This is the standard way to handle business workflows in distributed systems (orders, payments, bookings, signups). It comes from a 1987 paper by Garcia-Molina and Salem, and it found new life in the microservices era.

The problem sagas solve

A typical order workflow touches multiple services:

```
Place Order
└ reserve inventory (inventory service)
└ charge payment (payment service)
└ schedule shipment (shipping service)
└ send confirmation (notification service)
```

In a monolith with one database, you wrap this in a single transaction. Either all of it commits, or none of it does.

In microservices, each service has its own database. There's no single transaction that spans them. You face a few options:

- **Two-phase commit (2PC).** Coordinator-based protocol. Doesn't scale, blocks resources, and most modern DBs and message brokers don't support it cleanly.
- **Just do them in order and hope.** Inventory gets reserved, payment fails, customer never gets a refund. Inventory leaks forever.
- **Saga.** Each service commits its own local transaction. If a step fails, compensations roll back the previous ones.

Sagas trade strong consistency for eventual consistency. Done right, users never notice. Done wrong, the system silently leaks data.

The two flavors

Choreography

Each service reacts to events. No central coordinator. The saga emerges from a chain of pub/sub events.

```
OrderCreated event
└─ Inventory service reserves stock
└─ publishes InventoryReserved
```

```
InventoryReserved event
└─ Payment service charges card
└─ publishes PaymentCharged
```

```
PaymentCharged event
└─ Shipping service schedules
└─ publishes ShipmentScheduled
```

```
ShipmentScheduled event
└─ Notification service sends email
└─ publishes OrderConfirmed
```

If any step fails, it publishes a compensation event. Other services react and undo their work.

Pros:

- Loose coupling. Each service knows about its own events, not the workflow.
- No coordinator to deploy and manage.
- Natural fit for event-driven architectures.

Cons:

- The saga logic lives in nobody's head. To understand it, you have to trace events through several services.
- Hard to monitor. "Where is order 42 in the workflow right now?" needs a separate tracker.
- Cyclic dependencies sneak in. Service A reacts to B, B reacts to A.

Best for short sagas (3-4 steps), event-native systems, and teams that own related services.

Orchestration

A central orchestrator drives the saga. It calls each service, tracks state, decides what to do next.

```

[Orchestrator]
| call inventory.reserve(orderId)
▼
[Inventory service]
| ok
▼
[Orchestrator]
| call payment.charge(orderId)
▼
[Payment service]
| failure!
▼
[Orchestrator]
| call inventory.release(orderId)
▼
[Inventory service]
| ok
▼
[Orchestrator]
| mark saga as failed

```

The orchestrator owns the workflow. Services expose RPCs and don't know about each other.

Pros:

- Workflow is visible in one place. Easier to reason about, test, and monitor.
- Easier to add new steps or change the order.
- State is centralized. "Where is order 42?" is a database query.

Cons:

- The orchestrator becomes critical infrastructure. Single point of failure if not designed carefully.
- Services lose autonomy slightly. The orchestrator dictates the flow.
- Anti-pattern risk: the orchestrator becomes the new monolith.

Best for longer sagas, complex workflows with branching, and teams that prefer explicit control.

Picking one

Factor	Choose
Few steps, event-driven system	Choreography
Many steps, need visibility	Orchestration
Each service owns part of the flow	Choreography
One team coordinates a business process	Orchestration
Hard requirements on tracing and retries	Orchestration

Hybrid systems exist. Some workflows orchestrate the main path and use events for side effects.

Compensation actions

Each step needs a compensation that undoes it.

Step	Compensation
Reserve inventory	Release inventory
Charge payment	Refund payment
Schedule shipment	Cancel shipment
Send notification	Send cancellation notification
Create account	Mark account as deleted (or soft delete)

Some compensations are perfect inverses (release the inventory). Others are semantic inverses. You can refund a charge, but the customer's bank statement still shows both transactions.

Design rules for compensations:

- **Idempotent.** A compensation must be safe to call multiple times. Retries happen.
- **Reach backward only.** A “compensation” that pushes the saga forward isn't really a compensation. It undoes local work and nothing else.
- **Always possible.** If a step can't be compensated (sent an email, called a third party), you have a problem. Restructure the saga to put irreversible steps last, or accept the leak.
- **Resilient to “already compensated” state.** Calling refund on an already-refunded payment must succeed (or return an explicit “already done”).

Compensations themselves can fail. That's the worst case in saga design and needs careful handling (see “Failure modes” below).

A worked example

Order placement saga.

Happy path

```
1. createOrder(items)          -> Order(status=pending)
2. reserveInventory(orderId)   -> ok
3. chargePayment(orderId)      -> ok
4. scheduleShipment(orderId)   -> ok
5. markOrderCompleted(orderId) -> Order(status=completed)
```

Failure at step 3

```
1. createOrder(items)      -> Order(status=pending)
2. reserveInventory(orderId) -> ok
3. chargePayment(orderId)  -> FAILED (insufficient funds)
```

Saga reverses:

```
  releaseInventory(orderId) -> ok
  markOrderFailed(orderId)  -> Order(status=cancelled)
```

Failure at step 4

```
1. createOrder(items)      -> Order(status=pending)
2. reserveInventory(orderId) -> ok
3. chargePayment(orderId)  -> ok
4. scheduleShipment(orderId) -> FAILED (carrier API down)
```

Saga reverses:

```
  refundPayment(orderId)    -> ok
  releaseInventory(orderId) -> ok
  markOrderFailed(orderId)  -> Order(status=cancelled)
```

Compensations run in reverse order of execution.

Saga states

An orchestrated saga (and even a tracked choreography) usually has these states:

State	Meaning
STARTED	Saga kicked off, first step in progress
RUNNING	Working through steps
COMPENSATING	A step failed, running compensations
COMPLETED	All steps succeeded
COMPENSATED	Steps undone after a failure
FAILED	Saga ended in an unrecoverable state

Persist the state. After a crash, you need to know where to resume.

Idempotency in sagas

Every step and every compensation must be idempotent. Why:

- Network blips cause retries. The same step may be invoked multiple times.
- A crash mid-call can leave the orchestrator unsure whether the call succeeded.
- Compensations get retried when they fail.

Implementation patterns:

- **Idempotency keys on every call.** The saga ID plus step number makes a natural key.
- **Conditional updates.** `SET status = 'reserved' WHERE status = 'pending'`. Repeats are no-ops.
- **Sequence numbers.** Reject calls with a step number that's already been processed.

```
public class InventoryService {
    public void reserve(String sagaId, int stepNumber, ReserveRequest req) {
        // Idempotency check
        if (repo.hasProcessed(sagaId, stepNumber)) {
            return;
        }
        repo.reserve(req.itemId(), req.quantity());
        repo.markProcessed(sagaId, stepNumber);
    }
}
```

Failure modes

The interesting cases are everything except “happy path”.

Step failure (the easy case)

A step returns an error. Run compensations in reverse. Mark the saga as compensated.

Compensation failure (the hard case)

A compensation throws or times out. Now you have a partial rollback.

Options:

- Retry the compensation with backoff. Most transient failures resolve on retry.
- Alert on persistent compensation failure. A human needs to intervene.
- Move the saga to a “manual reconciliation needed” state. Don’t pretend it succeeded.

Never silently swallow a compensation failure. That’s the path to leaked inventory or double charges.

Orchestrator crash mid-saga

The orchestrator dies after step 2 ran but before step 3 was called.

Defenses:

- Persist saga state before each step. When the orchestrator restarts, it reads the state and resumes.
- Use a durable workflow engine (Temporal, Cadence). They handle this automatically.

Lost or duplicated messages (choreography)

In event-driven sagas, the message broker might deliver an event twice or lose one.

Defenses:

- Idempotent consumers. Each event is processed once thanks to idempotency keys.
- At-least-once delivery plus dedup. Standard pattern for most brokers (Kafka, SQS, RabbitMQ).
- Outbox pattern on the publishing side. Ensures the event is published if and only if the DB write committed.

Compensation needs data that no longer exists

You charged the customer, then the customer deleted their account. Now you need to refund, but the account record is gone.

Solution: avoid hard deletes for records referenced by sagas. Soft delete with retention long enough to cover all in-flight sagas.

Implementation patterns

Outbox pattern + saga

The outbox guarantees that events are published only when the local transaction commits. Critical for choreography sagas.

```
BEGIN;  
UPDATE inventory SET reserved = reserved + 5 WHERE id = 42;  
INSERT INTO outbox (event_type, payload) VALUES ('InventoryReserved', ...);  
COMMIT;
```

A separate process reads the outbox and publishes to the broker.

State machine

Model the saga explicitly as a state machine. Each step is a transition.

```
enum OrderSagaState {  
    PENDING,  
    INVENTORY_RESERVED,  
    PAYMENT_CHARGED,  
    SHIPMENT_SCHEDULED,  
    COMPLETED,  
    COMPENSATING_PAYMENT,  
    COMPENSATING_INVENTORY,  
    FAILED  
}
```

Easy to reason about, easy to visualize. Most workflow engines do this under the hood.

Event sourcing + saga

Store the saga as a sequence of events rather than a row in a table. Replaying the events reconstructs the state. Good for audit-heavy domains.

Workflow engines

For non-trivial sagas, use a workflow engine. They handle persistence, retries, timeouts, and crash recovery automatically.

Engine	Notes
Temporal	Open-source, popular, code-first. Workflows are normal Java/Go/Python code.
Cadence	Uber's predecessor to Temporal. Similar model.
AWS Step Functions	Managed, JSON state machine definitions. Tight AWS integration.
Camunda	BPMN-based. Strong in enterprise process workflows.
Azure Durable Functions	Code-first sagas in Azure.
Conductor (Netflix)	JSON workflow definitions, microservice-focused.

Temporal example (Java):

```

@WorkflowInterface
public interface OrderWorkflow {
    @WorkflowMethod
    void placeOrder(OrderRequest request);
}

public class OrderWorkflowImpl implements OrderWorkflow {
    private final InventoryActivities inventory =
        Workflow.newActivityStub(InventoryActivities.class, options);
    private final PaymentActivities payment =
        Workflow.newActivityStub(PaymentActivities.class, options);
    private final ShippingActivities shipping =
        Workflow.newActivityStub(ShippingActivities.class, options);

    @Override
    public void placeOrder(OrderRequest request) {
        Saga saga = new Saga(new Saga.Options.Builder().build());
        try {
            inventory.reserve(request);
            saga.addCompensation(() -> inventory.release(request));

            payment.charge(request);
            saga.addCompensation(() -> payment.refund(request));

            shipping.schedule(request);
            saga.addCompensation(() -> shipping.cancel(request));
        } catch (Exception e) {
            saga.compensate();
            throw e;
        }
    }
}

```

Temporal's Saga helper records each compensation as you add it. On failure, `compensate()` runs them in reverse automatically.

Best practices

- **Make every step idempotent.** Network and orchestrator retries are inevitable.
- **Make compensations idempotent.** They'll get retried too.
- **Put irreversible steps last.** Send confirmation emails after the order is final.
- **Persist saga state before each step.** Recovery depends on it.
- **Use a workflow engine for non-trivial sagas.** Rolling your own state, persistence, retries, and timeouts is a long road.
- **Match the flavor to the situation.** Choreography for simple sagas, orchestration for complex ones.
- **Monitor saga health.** Track saga duration, failure rate, compensation rate. Long-running or stuck sagas indicate problems.

- **Test failure paths.** Inject failures at each step. Verify compensations actually run.
- **Document the workflow.** A diagram of the saga steps and compensations should live next to the code.
- **Avoid hard-deleting data referenced by sagas.** Soft delete with a retention window.

Common gotchas

- **Forgetting that compensations can fail.** Plan for it. A retry strategy plus a manual escape hatch.
- **Non-idempotent steps.** A retry double-charges the customer or double-reserves inventory.
- **Compensation that advances instead of retreats.** A “compensation” that creates a credit note while leaving the original charge in place doesn’t actually undo anything. Make sure compensations reverse the effect.
- **Side effects without compensations.** Sending an email is hard to undo. Save it for the last step, or have a “we made a mistake” follow-up flow.
- **Choreography sprawl.** What started as 3 events becomes a 20-event tangle that nobody fully understands. Migrate to orchestration when complexity grows.
- **Treating the orchestrator as a god service.** Keep it focused on workflow control. Business logic stays in the services.
- **Saga state inconsistent with service state.** If the saga thinks step 3 succeeded but the service has no record of it, you’ll compensate something that didn’t happen. Audit periodically.
- **No timeout on a step.** A hanging external call leaves the saga stuck forever. Always set deadlines.
- **Distributed deadlocks via choreography.** Service A reacts to B, B reacts to A. Both end up waiting. Watch the event graph.
- **Versioning workflows.** A long-running saga started under workflow v1 may finish under v2. Workflow engines handle this. Hand-rolled state machines often don’t.
- **Cross-saga interactions.** Two sagas touch the same row. Use optimistic locking or per-resource serialization to avoid races.

Quick reference

Concept	Key fact
Saga	Sequence of local transactions with compensating actions
Compensation	Undoes a previously-completed step
Choreography	Event-driven, no central coordinator
Orchestration	Central service drives the workflow
Idempotency	Required for every step and every compensation
Outbox pattern	Ensures events are published with the DB commit
Workflow engine	Use one for non-trivial sagas (Temporal, Cadence, Step Functions)
State machine	Explicit model of saga progress
Reverse order	Compensations run in reverse of original steps
Compensation failure	The hard case. Retry, alert, never swallow.
Irreversible steps	Push them to the end of the saga
Soft delete	Don't hard-delete data still referenced by in-flight sagas